

CODE:

```
# Install required libraries
```

```
!pip install -q transformers datasets nltk pandas matplotlib scikit-learn keybert sentence-  
transformers wordcloud
```

```
import pandas as pd
```

```
import re
```

```
import nltk
```

```
import matplotlib.pyplot as plt
```

```
from nltk.corpus import stopwords
```

```
from nltk.stem import PorterStemmer
```

```
from transformers import pipeline
```

```
from keybert import KeyBERT
```

```
import pandas as pd          # For handling CSV and DataFrame operations
```

```
import re                    # For regular expressions (cleaning text)
```

```
import nltk                  # Natural Language Toolkit for text preprocessing
```

```
import matplotlib.pyplot as plt # To create visual charts
```

```
from nltk.corpus import stopwords # List of common stopwords (like "is", "the", etc.)
```

```
from nltk.stem import PorterStemmer # For reducing words to their root (stemming)
```

```
from transformers import pipeline # Hugging Face pipeline for AI models
```

```
from keybert import KeyBERT    # For keyword extraction using BERT
```

```
# NLTK stopwords
```

```
nltk.download('stopwords')
```

```
# Setup stopwords and stemmer
```

```
stop_words = set(stopwords.words('english'))
```

```
stemmer = PorterStemmer()
```

```

# dataset

df = pd.read_csv('/content/twitter_validation_new.csv')

print("✅ Columns found:", df.columns)

print(df.head())


# Clean text function

def clean_text(text):

    text = re.sub(r"http\S+|www\S+|https\S+", "", str(text)) # Remove URLs

    text = re.sub(r"@w+|\#", "", text) # Remove mentions & hashtags

    text = re.sub(r"[^a-zA-Z]", " ", text) # Remove non-letters

    text = text.lower()

    tokens = text.split()

    tokens = [stemmer.stem(word) for word in tokens if word not in stop_words]

    return " ".join(tokens)


df['cleaned_text'] = df['Text'].apply(clean_text)


# Sentiment Analysis

sentiment_pipeline = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")


def predict_sentiment(text):

    result = sentiment_pipeline(text)[0]

    return result['label'], result['score']


df[['predicted_label', 'confidence']] = df['cleaned_text'].apply(lambda x:
pd.Series(predict_sentiment(x)))

df['sentiment_emoji'] = df['predicted_label'].map({'POSITIVE': '😊', 'NEGATIVE': '😞'})


# Emotion Detection with categories & emojis

emotion_pipeline = pipeline("text-classification", model="j-hartmann/emotion-english-distilroberta-base", return_all_scores=False)

```

```
# Map model emotion labels
```

```
emotion_to_category_emoji = {
```

```
    'joy':    ('joy', '😊'),
```

```
    'sadness': ('sad', '😞'),
```

```
    'neutral': ('neutral', '😐'),
```

```
    'anger':   ('hurt', '😡'),
```

```
    'fear':    ('hurt', '😱'),
```

```
}
```

```
def map_emotion_to_category_emoji(model_emotion_label):
```

```
    label = model_emotion_label.lower()
```

```
    return emotion_to_category_emoji.get(label, ('neutral', '😐'))
```

```
def predict_custom_emotion(text):
```

```
    result = emotion_pipeline(text)[0]
```

```
    category, emoji = map_emotion_to_category_emoji(result['label'])
```

```
    confidence = result['score']
```

```
    return category, emoji, confidence
```

```
df[['custom_emotion', 'emoji', 'emotion_confidence']] = df['cleaned_text'].apply(
```

```
    lambda x: pd.Series(predict_custom_emotion(x))
```

```
)
```

```
# Toxicity Detection
```

```
toxicity_pipeline = pipeline("text-classification", model="unitary/toxic-bert")
```

```
def predict_toxicity(text):
```

```
    result = toxicity_pipeline(text)[0]
```

```
    return result['label'], result['score']
```

```
df[['toxicity', 'toxicity_score']] = df['cleaned_text'].apply(lambda x: pd.Series(predict_toxicity(x)))
```

```
# Keyword Extraction
```

```
kw_model = KeyBERT()
```

```
df['keywords'] = df['cleaned_text'].apply(lambda x: kw_model.extract_keywords(x, top_n=3))
```

```
# Preview sample
```

```
print("\n🇮🇹 Sample predictions with custom emotions and emojis:")
```

```
print(df[['Text', 'custom_emotion', 'emoji', 'emotion_confidence', 'toxicity', 'keywords']].head())
```

```
# Summary counts
```

```
emotion_counts = df['custom_emotion'].value_counts()
```

```
print(f"\nTotal Tweets: {len(df)}")
```

```
print("Emotion counts:")
```

```
print(emotion_counts)
```

```
# Bar chart
```

```
plt.figure(figsize=(8, 5))
```

```
emotion_counts.plot(kind='bar', color='skyblue')
```

```
plt.title('Custom Emotion Distribution')
```

```
plt.ylabel('Number of Tweets')
```

```
plt.xlabel('Emotion')
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.show()
```

```
# Pie chart for custom emotions (optional)
```

```
plt.figure(figsize=(7, 7))
```

```
plt.pie(emotion_counts, labels=[f'{emo} {cnt}' for emo, cnt in emotion_counts.items()],
```

```
       autopct='%1.1f%%', startangle=90, colors=plt.cm.Paired.colors)
```

```
plt.title('Custom Emotion Distribution')
```

```
plt.axis('equal')
```

```
plt.show()
```

```
# ✅ Save results to CSV
```

```
df.to_csv('tweet_analysis_results.csv', index=False)
```

```
print("✅ Results saved to 'tweet_analysis_results.csv'")
```

OUTPUT:

Text custom_emotion emoji \

0	I mentioned on Facebook that I was struggling ...	joy	😊
1	BBC News - Amazon boss Jeff Bezos rejects clai...	hurt	😞
2	@Microsoft Why do I pay for WORD when it funct...	neutral	😐
3	CSGO matchmaking is so full of closet hacking,...	neutral	😐
4	Now the President is slapping Americans in the...	hurt	😞

emotion_confidence toxicity \

0	0.773366	toxic
1	0.696507	toxic
2	0.904423	toxic
3	0.642802	toxic
4	0.588336	toxic

keywords

0	[(motiv, 0.4362), (grandma, 0.427), (facebook, ...
1	[(bezo, 0.402), (jeff, 0.3708), (drug, 0.3367)]
2	[(chromebook, 0.5193), (pay, 0.365), (word, 0....
3	[(matchmak, 0.5798), (csgo, 0.5563), (game, 0....
4	[(unlaw, 0.4335), (act, 0.4121), (presid, 0.38...

Total Tweets: 1000

Emotion counts:

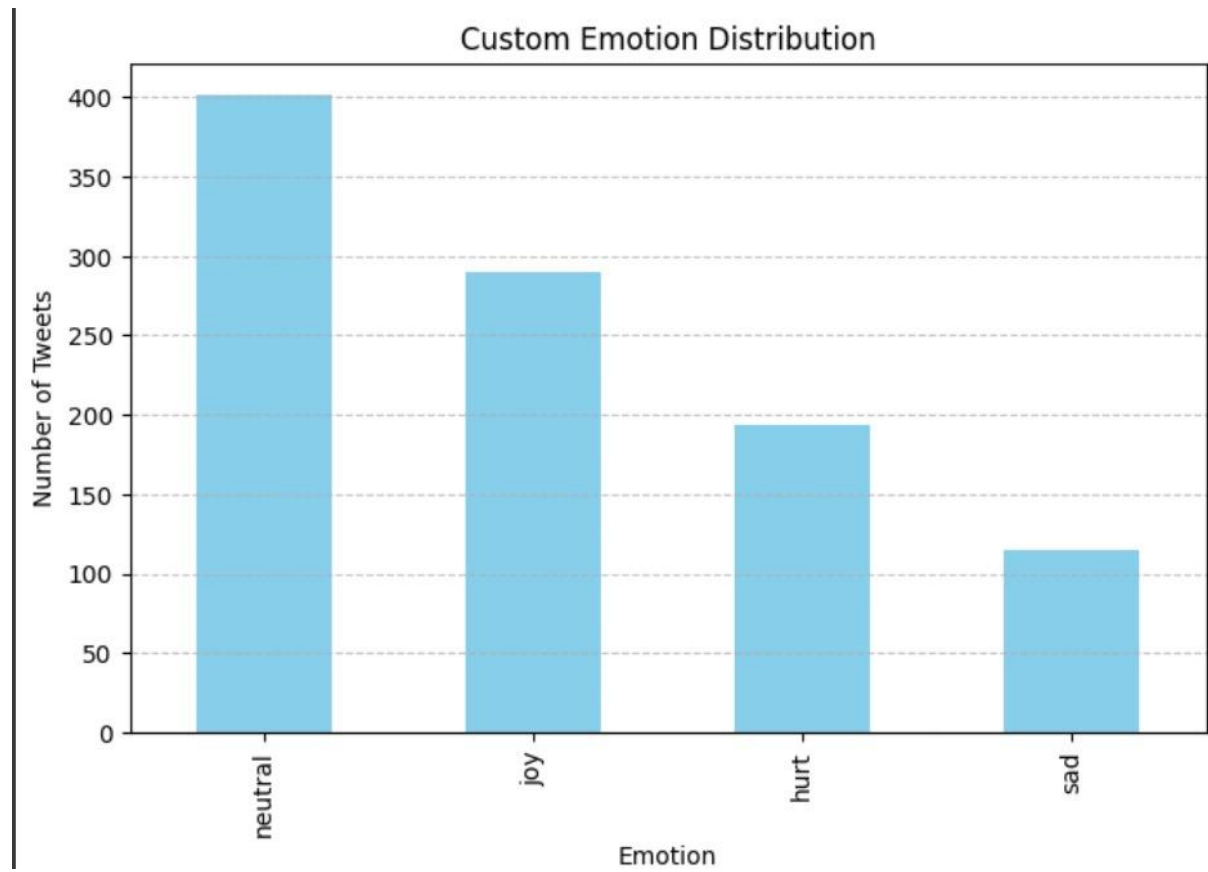
custom_emotion

neutral 401

joy 290

hurt 194

sad 115



Custom Emotion Distribution

